

دانشگاه صنعتی امیرکبیر

(پلی‌تکنیک تهران)

دانشکده مهندسی کامپیوتر

پروژه دوم درس سیستم‌های نهفته و بی‌درنگ

آشنایی با سیستم‌عامل FreeRTOS

استاد:

دکتر فربه

ترم بهار ۱۴۰۴

طراح پروژه: مهیار اسماعیلی

مقدمه

احتمالا تا به حال برای برنامه‌نویسی میکروکنترلرها بخش زیادی از کد برنامه خود را در یک حلقه بی‌نهایت (while(1)) یا همان تابع loop() در آردوینو) قرار می‌دادید و با استفاده از متغیرهای گلوبال و وقفه‌ها سعی می‌کردید جریان برنامه را کنترل کنید. با پیچیده‌تر شدن سیستم‌های نهفته، کنترل کردن عملیات‌هایی مثل خواندن از سنسورها، مدیریت کردن ارتباطات و منطق برنامه با روش گفته شده بسیار پیچیده می‌شود.

FreeRTOS یک سیستم‌عامل بلادرنگ[1] متن‌باز است که به طور ویژه برای سخت‌افزارهای محدودی مثل میکروکنترلرها طراحی شده است. در این پروژه قرار است چند تسک ساده برای این سیستم‌عامل تعریف کنیم تا تفاوت مدل برنامه‌نویسی به هنگام استفاده از یک RTOS به جای حلقه بینهایت را درک کنیم.

سوالات خود را می‌توانید از طریق آیدی @mahyar_esm در بله یا @mahyar_esmaeeli در تلگرام مطرح کنید.

[1] Real-Time Operating System

راه‌اندازی ابزارها

نرم‌افزار Proteus

اگر شبیه‌ساز Proteus و یا کتابخانه Arduino آن را ندارید می‌توانید از لینک‌های زیر استفاده کنید:

- Proteus
- کتابخانه Arduino برای Proteus

در صفحه دانلود کتابخانه راهنمای اضافه کردن آن به Proteus قرار داده شده اما اگر به مشکلی برخوردید می‌توانید سوال کنید.

کتابخانه FreeRTOS برای Arduino

از لینک زیر پروژه را کلون کنید یا از فایل zip. که برایتان قرار داده شده استفاده کنید:

- Arduino FreeRTOS Library

پس از extract کردن، تمام پوشه‌های داخل *FreeRTOS-Arduino-master/libraries* (شامل *FreeRTOS_AVR* و غیره) را در مسیر *C:Users/<username>/Documents/Arduion/libraries* کپی کنید (اگر پوشه *libraries* در این مسیر وجود نداشت خودتان آن را بسازید).

ArdinoIDE را باز کنید و از طریق منو $Sketch \rightarrow Include\ Library \rightarrow FreeRTOS_AVR$ کتابخانه را اضافه کنید. سپس پروژه را بیلد کنید.

کار با FreeRTOS

در این قسمت نحوه کار کردن با FreeRTOS و تعدادی از توابع مورد نیاز را به طور مختصر توضیح می‌دهیم.

FreeRTOS Library

قبل از شروع به انجام هر کاری باید ابتدا کتابخانه FreeRTOS را اضافه کنید. برای Arduino Uno که از یک پردازنده AVR استفاده می‌کند مطابق شکل ۱ کتابخانه را اضافه کنید:

```
1   #include <FreeRTOS_AVR.h>
2
```

شکل ۱- اضافه کردن کتابخانه

Task Function

برای کار با FreeRTOS باید ابتدا Taskها را تعریف کنیم. مهم‌ترین مرحله در تعریف Task، تعریف Task Function یا Task Code است. Task Function یک تابع است که منطق Task در آن تعریف می‌شود. یک نمونه از Task Function در شکل ۲ آمده است.

```
 99  const uint8_t LED_PIN = 12;
100  void vTask1(void *arg) {
101    //setup
102    pinMode(LED_PIN, OUTPUT);
103    bool state = true;
104
105    //loop
106    while (1) {
107      vTaskDelay(200);
108      digitalWrite(LED_PIN, state);
109      state = !state;
110    }
111  }
```

شکل ۲- نمونه Task Function

مطابق پیاده‌سازی تابع، این Task قرار است یک LED را روشن و خاموش کند. ساختار یک Task Function کاملا مشابه یک پروژه آردوینو است. همانطور که می‌بینید قسمت setup و loop دارد. در واقع انگار که هر Task یک پروژه آردوینو است که همزمان با بقیه اجرا می‌شود.

قسمتی که در حلقه بی‌نهایت قرار گرفته آن قسمتی است که همواره و هر وقت که scheduler پردازنده را در اختیار Task1 قرار می‌دهد اجرا می‌شود.

نکات:

- مبنای زمان در سیستم‌عامل FreeRTOS واحدی به نام tick است. این که هر tick معادل چند میلی‌ثانیه باشد را می‌توان تغییر داد اما به صورت پیش‌فرض معمولا یک tick حدودا یک میلی‌ثانیه است.
- با گذشت هر tick، یک وقفه تایمر ایجاد می‌شود که در ISR آن کد scheduler اجرا می‌شود.
- تابع vTaskDelay() به میزان مقداری که به آن پاس داده شده است تاخیر ایجاد می‌کند. مثلا اگر مقدار پاس داده شده ۲۰۰ باشد، تسک حداقل تا ۲۰۰ تیک بعد اجرا نخواهد شد.
- تابع vTaskDelay() مکانیزمی متفاوت از delay()ای که شما با آن آشنا هستید دارد. در واقع با استفاده از این تابع، Task به حالت Blocked می‌رود و scheduler می‌تواند Taskهای دیگر داخل Ready Queue را به حالت Running ببرد. این در حالی است که تابع delay مبتنی بر busy-waiting است. در کل توصیه می‌شود زمانی که با FreeRTOS کار می‌کنید از تابع delay() استفاده نکنید؛ حتی اگر واقعا به busy-waiting نیاز دارید.
- یک Task Function همیشه خروجی از نوع void و ورودی از نوع void * دارد.
- نوع void * در زبان C یک نوع general است. شما می‌توانید ورودی از این نوع را به نوع اصلی آن cast و از آن استفاده کنید. در این پروژه نیازی به استفاده از این پارامتر نخواهید داشت.
- دلیل نام‌گذاری تابع به شکل vTask1 به قراردادهای نام‌گذاری FreeRTOS برمی‌گردد (نوع خروجی تابع باید با حرف کوچک در ابتدای نام تابع ظاهر شود). نیازی به رعایت این قراردادها نیست. در صورتی که علاقه‌مند به کسب اطلاعات بیشتر بودید می‌توانید به داکیومنت رسمی FreeRTOS که برایتان قرار داده می‌شود مراجعه کنید.

ساختن یک Task

پس از تعریف Task Functionها به سراغ تعریف تسک‌ها می‌رویم که در واقع برای معرفی تسک‌ها به scheduler انجام می‌شود. شکل ۳ را ببینید.

```
void setup() {
  portBASE_TYPE s;
  s = xTaskCreate(vTask1, "optional name", 128, NULL, 1, NULL);
  if (s != pdPASS) {
    Serial.println("task creation failed");
    while (1);
  }
  vTaskStartScheduler();
  Serial.println("Scheduler failed");
  while(1);
}
```

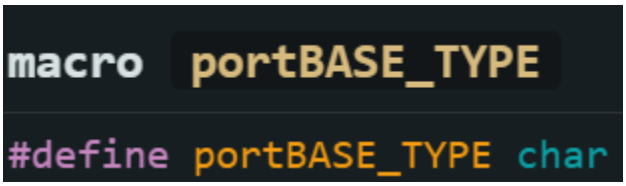
شکل ۳- ساختن Task

تابع ()xTaskCreate برای ما یک تسک می‌سازد و خروجی آن نتیجه عملیات است. پارامترهای این تابع به ترتیب به شکل زیر هستند:

۱. اشاره‌گر به Task Function (نام تابعی که تعریف کردید بدون پرانتز قرار دهید).
۲. نامی که برای تسک می‌گذارید. می‌تواند NULL باشد.
۳. مقدار حافظه‌ای که می‌خواهید به تسک اختصاص دهید (بر حسب word). برای این پروژه می‌توانید ۱۲۸ قرار دهید.
۴. پارامترهای تسک: این همان چیزی است که Task Function شما به عنوان پارامتر ورودی دریافت می‌کند (از طریق آرگومان void *args). می‌تواند NULL باشد.
۵. اولویت تسک (هرچه بیشتر با اولویت‌تر).
۶. اشاره‌گر به تسک، اگر بعدا بخواهید در تسک دیگر کاری با این تسک بکنید، مثلا اولویت آن را تغییر بدهید یا آن را Suspend کنید از آن استفاده می‌شود. در این پروژه نیازی به آن نخواهید داشت. می‌توانید NULL قرار دهید.

نکات:

- زمان‌بندی FreeRTOS از نوع Fixed-Priority Scheduling است (مشابه الگوریتم RM) یعنی به هنگام تعریف تسک‌ها برای آن‌ها اولویت تعیین شده و زمان‌بندی هم بر حسب همین اولویت‌ها انجام می‌شود. بنابراین الگوریتم‌هایی مثل EDF که از نوع Dynamic-Priority Scheduling هستند به صورت پیش‌فرض قابل انجام نیستند و در صورت نیاز خودتان باید آن‌ها را پیاده‌سازی کنید.
- ساختن تسک‌ها توسط تابع xTaskCreate() را در داخل تابع setup() آردوینو انجام دهید.
- نوع portBASE_TYPE بسته به معماری پردازنده متفاوت است. مثلا برای پردازنده‌های ۳۲بیتی معمولا یک Integer علامت‌دار ۳۲بیتی است. برای Arduino Uno که از یک پردازنده ۸بیتی استفاده می‌کند، این نوع به عنوان char تعریف شده است (در صورتی که کرسر ماوس را روی این نوع قرار دهید می‌توانید تعریف آن را مطابق شکل ۴ مشاهده کنید).



شکل ۴- نوع portBASE_TYPE

- خروجی تابع xTaskCreate() از نوع portBASE_TYPE است و مشخص می‌کند که ساختن تسک موفقیت آمیز بوده یا نه.
- پس از ساختن تسک‌ها باید به scheduler بگویید کار خود را شروع کند که این کار با فراخوانی تابع vTaskStartScheduler() انجام می‌شود. این فراخوانی هم در تابع setup() انجام دهید. همانطور که در شکل ۲ می‌بینید، در صورتی که همه چیز موفقیت آمیز باشد این تابع هیچوقت return نمی‌کند (پس عبارت "Scheduler Failed" نباید چاپ شود).

در اینجا می‌توانید کد فعلی حاصل از این مراحل را داخل شبیه‌ساز تست کنید تا ببینید همه چیز درست کار می‌کند یا خیر. در منو گزینه $Sketch \rightarrow Export\ Complied\ Library$ را انتخاب کنید تا فایل hex. در پوشه *build* و در همان مسیری که کد پروژه قرار گرفته برایتان ساخته شده و بتوانید داخل برد شبیه‌ساز بارگذاری کنید. سپس یک LED به پین ۱۲ برد متصل کرده و نتیجه را ببینید.

قابلیت‌های سیستم‌عامل FreeRTOS

سیستم‌عامل FreeRTOS دارای قابلیت‌های بسیار زیادی برای همگام‌سازی تسک‌ها (مثل سمافورها، mutex و TaskNotification) و انتقال اطلاعات میان آن‌ها (با استفاده از Queueها) است و همچنین توابعی برای کار با ISRها نیز دارد که در این پروژه به آن‌ها پرداخته نشده است. در صورتی که علاقه‌مند هستید می‌توانید با مراجعه به داکیومنت‌های رسمی FreeRTOS یا مراجع دیگر اطلاعات بیشتری درباره این سیستم‌عامل کسب کنید.

تحویل پروژه

برای تحویل این پروژه کافی است سه تسک تعریف کنید که هرکدام با استفاده از تابع ()vTaskDelay یک LED را با دوره تناوب‌های (منظور دوره تناوب تسک است نه خود LEDها) زیر روشن و خاموش می‌کنند:

- تسک اول با دوره تناوب ۱۰۰ میلی‌ثانیه
- تسک دوم با دوره تناوب ۵۰۰ میلی‌ثانیه
- تسک سوم با دوره تناوب ۱۰۰۰ میلی‌ثانیه

در نهایت خروجی را داخل شبیه‌ساز مشاهده کنید.

سپس به دو سوال زیر پاسخ دهید:

۱. اگر می‌خواستید همین کار را بدون استفاده از یک RTOS و با همان روش قبلی که با آن آشنا هستید انجام دهید باید چکار می‌کردید (نیازی به نوشتن کد نیست؛ روش را به صورت کلی توضیح دهید)؟
۲. با توجه به توضیحی که از عملکرد ()vTaskDelay دادیم، به نظر شما آیا استفاده از این تابع برای اجرای تسک‌ها به صورت پریودیک انتخاب خوبی است؟ مشکل این تابع چیست؟
درباره تابع ()vTaskDelayUntil اندکی تحقیق کنید و ببینید که تفاوت آن با ()vTaskDelay چیست و چطور مشکل ()vTaskDelay را حل می‌کند.

در نهایت یک فایل zip. حاوی فایل‌های زیر را در سامانه کورسز در بخش مربوطه آپلود کنید.

- فایل ino. حاوی کدهای پروژه
- یک ویدئو کوتاه از عملکرد کد داخل شبیه‌ساز (خاموش و روشن شدن LEDها)
- فایل pdf حاوی پاسخ به دو سوال پرسیده شده